MCTR911 Robotics Programming — Milestone 01

Literature Review: Sorting Boxes by Colour with a Low-Cost Robot Arm

Team: [team 06] · Members: [Youssef hazem – ID:58-2769], [maged emil – ID:58-3830], [tony hany – ID:58-6506], [anas mohamed – ID:58-2897], [ahmed adham – ID:58-3529]

1. What we are building

We are building a robot cell that **sorts boxes by colour**. Three boxes (red, green and blue) wait on a conveyor in front of a Low-Cost Robot Arm (Koch v1.1). The robot picks up each box with its built-in parallel gripper and drops it into the bin of the same colour. The bins are on the robot's left, right and behind it. To keep the project simple, the conveyor does not move and the box positions are read directly from the simulator (no camera).

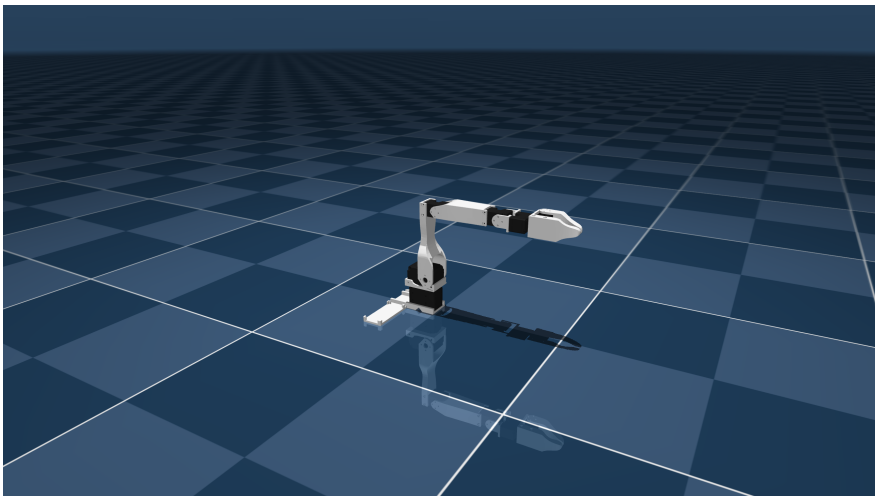


Fig. 1 – The Low-Cost Robot Arm (Koch v1.1) from MuJoCo Menagerie [11], shown in MuJoCo. The conveyor, boxes and bins will be added around it.

2. How this problem is normally solved

Kinematics. The standard way to model a robot arm is the Denavit–Hartenberg (DH) convention: a frame is placed on each joint, and four parameters per link give the transform from one frame to the next. Multiplying these transforms gives the forward kinematics (joint angles → tool position) [1]. Inverse kinematics (tool position → joint angles) is solved with a geometric decoupled approach: choose the base yaw, solve the planar two-link shoulder–elbow problem with the cosine rule [2], then set the wrist joints for the desired approach direction. The Jacobian also links joint velocities to tool velocities [3], [4].

Motion planning. Long moves are usually planned in joint space with cubic or quintic polynomials, so each joint starts and stops smoothly. Short moves where the path matters, like going straight down to a part, are planned as a straight line in task space and converted to joint angles with inverse kinematics [3], [4].

Control. The most common controller is a separate PD or PID loop on each joint. It is simple and works well at moderate speeds. Computed-torque control uses the robot's dynamic model to cancel gravity and inertia, so it follows the planned path more accurately [3], [4].

Simulation. The system is normally tested in a simulator before a real robot is used. Common choices are Gazebo [5], PyBullet [6] and MuJoCo [7]; MuJoCo is fast and handles contact between objects well. ROS 2 is the standard software layer that connects the control code to the robot or

simulator by sending messages on named topics [8].

3. The robot we picked and why

We picked the **Low-Cost Robot Arm (Koch v1.1)**, an open-source tabletop manipulator originally designed by Alexander Koch [9] and updated by the HuggingFace LeRobot team [10], with 5 positioning joints and a built-in gripper. Why this robot:

1. **5 positioning DoF,** which is inside the 4–6 DoF range needed for pick-and-place tasks.
2. **Derivable DH parameters [9], [10],** from the public URDF, so we can build and check our own DH table.
3. **A known geometric IK [2],** with two elbow solutions (elbow-up and elbow-down), so we can verify our IK results.
4. **A ready-made MuJoCo model** (low_cost_robot_arm) from MuJoCo Menagerie [11], so we do not have to build the model ourselves.
5. **Built-in parallel gripper:** no separate end-effector is needed, and the 330 mm reach means all bins must be placed close to the base, which we will verify in simulation.

4. Draft project flow

One sorting cycle (repeated until the conveyor is empty):

1. **Home:** wait in a safe pose and read the position and colour of the next box.
2. **Approach:** move to 5 cm above the box, then go straight down to it.
3. **Grasp and lift:** close the gripper and lift the box straight up.
4. **Move:** swing smoothly to a point above the bin with the same colour.
5. **Place and return:** open the gripper, lower the box, and go back home.

Plan: M02 – frames, DH table, forward and inverse kinematics, joint sliders in MuJoCo, start of the GUI. M03 – velocity and acceleration kinematics (Jacobian), checked against the simulator, and the full tabletop environment. M04 – joint-space and task-space trajectories, gripper control, and the complete sorting cycle.

References

[1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed., Pearson, 2018.

[2] D. Pieper, “The kinematics of manipulators under computer control,” PhD thesis, Stanford University, 1968.

[3] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.

[4] M. W. Spong, S. Hutchinson, M. Vidyasagar, *Robot Modeling and Control*, 2nd ed., Wiley, 2020.

[5] N. Koenig, A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” *Proc. IEEE/RSJ IROS*, pp. 2149–2154, 2004.

[6] E. Coumans, Y. Bai, “PyBullet, a Python module for physics simulation for games, robotics and machine learning,” 2016–2021. https://pybullet.org

[7] E. Todorov, T. Erez, Y. Tassa, “MuJoCo: A physics engine for model-based control,” *Proc. IEEE/RSJ IROS*, pp. 5026–5033, 2012.

[8] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.

[9] A. Koch, “Low-Cost Robot Arm,” GitHub repository, 2023. https://github.com/AlexanderKoch-Koch/low_cost_robot

[10] J. Moss et al., “Koch v1.1 — HuggingFace LeRobot,” GitHub repository, 2024. https://github.com/jess-moss/koch-v1-1

[11] K. Zakka, Y. Tassa, and MuJoCo Menagerie contributors, “MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo,” 2022. https://github.com/google-deepmind/mujoco_menagerie